

Reviewer Pack — Erdős-Szekeres Theorem Based Lower Bound for Resolution Leng...

Entry 0f6ea0ea5bc1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 13:00:08 UTC

Erdős-Szekeres Theorem Based Lower Bound for Resolution Length on Tseitin Formulas

Entry ID: 0f6ea0ea5bc1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 13:00:08 UTC

1. Conjecture

Field A (mathematical branch): Number Theory: Erdős-Szekeres Theorem **Field B** (complexity object): Complexity Theory: Resolution Length of Tseitin Formulas

Statement:

[‘For any Tseitin formula constructed from an n -vertex graph G , the Resolution length for refuting the formula is lower-bounded by a function of the form $\Omega(2^{n/2} * \log(n))$.’, ‘Specifically, there exists a constant $C > 0$ such that for all Tseitin formulas on graphs with at least n vertices, their Resolution length satisfies $L_Resolution(G) \geq C * 2^{n/2} * \log(n)$.’, ‘This lower bound is independent of the specific graph G but holds for all such graphs.’]

Rationale (proposer’s reasoning):

[‘The Erdős-Szekeres theorem can be used to find long increasing subsequences within a set, which has implications for the structure of Tseitin formulas.’, ‘Such long subsequences could correspond to clauses that are hard to resolve in Resolution refutations, leading to an exponential lower bound on the resolution length.’, ‘This conjecture aims to leverage number theory to provide a new perspective on the complexity of reasoning about Boolean satisfiability.’]

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 30ef52bfa10b8dbe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all n -vertex graphs, the mean Resolution length $L_Resolution(G)$ is within a factor of 2 of $C * 2^{n/2} * \log(n)$, where C is a constant greater than zero, and falsified if any seed produces a metric with an $L_Resolution(G)$ outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        graph = {i: set() for i in range(n)}
        edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
        random.shuffle(edges)
        m = min(2 * (n - 1), len(edges))
        for i in range(m):
            u, v = edges[i]
            graph[u].add(v)
            graph[v].add(u)
        return graph

    def tseitin_formula(graph):
        n = len(graph)
        literals = [f'x{i}' for i in range(n)]
        clauses = []

        # Each vertex must be connected to at least one other vertex
        for i in range(n):
            if not graph[i]:
                continue
            clause = [literals[i]]
            for j in graph[i]:
                clause.append(f'¬{literals[j]}')
            clauses.append(clause)

        # Each edge is represented by a unique literal
        for u, v in [(i, j) for i in range(n) for j in range(i + 1, n)]:
```

```

    edge_literal = f'e{u}{v}'
    clauses.append([edge_literal, f'-x{u}', f'-x{v}'])
    clauses.append([f'~{edge_literal}', f'x{u}', f'x{v}'])

    return literals, clauses

def resolution_length(clauses):
    n = len(clauses)
    unit_clauses = {i: [] for i in range(n)}
    for i, clause in enumerate(clauses):
        if len(clause) == 1:
            unit_clauses[i].append(clause[0])

    resolvents = []
    while True:
        new_resolvents = set()
        for i in range(n):
            for j in range(i + 1, n):
                if not (set(unit_clauses[i]) & set(unit_clauses[j])):
                    continue
                common_lit = list(set(unit_clauses[i]) & set(unit_clauses[j]))[0]
                resolvent = [lit for lit in unit_clauses[i] if lit != common_lit] + \
                    [lit for lit in unit_clauses[j] if lit != f'~{common_lit}']
                resolvents.add(tuple(sorted(resolvent)))
        if not new_resolvents:
            break
        unit_clauses[n] = list(new_resolvents)
        n += 1

    return len(unit_clauses[n - 1])

def erdos_szekeres_lower_bound(n):
    C = 1.0 # Constant C > 0, chosen for simplicity
    return C * 2**(n / 2) * math.log(n)

n_values = [5, 10, 15, 20, 30, 40]
total_length = 0
instances_tested = 0

for n in n_values:
    graph = generate_random_graph(n)
    literals, clauses = tseitin_formula(graph)
    length = resolution_length(clauses)
    lower_bound = erdos_szekeres_lower_bound(n)

    total_length += length
    instances_tested += 1

    if length < lower_bound / 2 or length > 2 * lower_bound:
        return {
            "metric_name": "Resolution Length",
            "metric_value": length,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": f"n={n}, L_Resolution(G)={length}, Lower Bound={lower_bound}"
        }

```

```

mean_length = total_length / instances_tested
return {
    "metric_name": "Resolution Length",
    "metric_value": mean_length,
    "instances_tested": instances_tested,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    support_count = sum(1 for r in results if r["conjecture_holds"])
    support_fraction = support_count / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

246985'}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Resolution Length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
RESULT: FALSIFIED counterexample="n=5, L_Resolution(G)=0, Lower Bound=9.104355694246985" first_failing_seed=246985

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test has only tested up to $n=5$, which is too small to confirm the conjecture’s validity over a wide range of graph sizes. Additionally, the counterexample provided ($n=5$, $L_Resolution(G)=0$) suggests that the lower bound might not hold for all graphs, indicating a potential failure mode in the metric definition or the construction of Tseitin formulas.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for $n=5$, the Resolution length $L_Resolution(G)$ is 0, which is below the lower bound of 9.104355694246985 as calculated by t | next: Further investigation is needed to determine if the lower bound holds for larger graphs and to identify any potential issues with the metric definition or Tseitin formula construction.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15641
2	preregistration	ollama_remote	glm4:latest	0	0	10119
3	novelty	ollama_remote	glm4:latest	0	0	8955
4	novelty	ollama_remote	glm4:latest	0	0	8716
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13777
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10690
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11161
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13037
9	critic	ollama_remote	glm4:latest	0	0	12592
10	judge	ollama_remote	glm4:latest	0	0	11953

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 116641 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0f6ea0ea5bc1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f6ea0ea5bc1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0f6ea0ea5bc1.tar.gz` (if generated)